UNIVERSIDAD DE COSTA RICA

ESCUELA DE CIENCIAS DE LA COMPUTACIÓN E INFORMÁTICA

PROYECTO INTEGRADOR DE SISTEMAS OPERATIVOS Y REDES DE COMUNICACIÓN

Proyecto del curso - Etapa II

Protocolo de comunicación y conexión

DOCENTE:

Prof. Maeva Murcia Meléndez

Prof. Francisco Arroyo

Estudiantes:

Grupo 2

I Semestre - 2026

Protocolo de comunicación y conexión - Proyecto: Etapa 2

I. Introducción

El presente protocolo define un conjunto de reglas para el intercambio de información entre componentes dentro de un sistema de consulta de figuras construidas con piezas de lego. El objetivo principal del protocolo es permitir que los clientes puedan:

- Consultar las figuras disponibles en el sistema.
- Obtener las piezas necesarias para construir una figura específica.

El protocolo sigue una arquitectura lógica de tipo cliente–intermediario–servidor, en la cual cada componente cumple un rol claramente definido y la comunicación se realiza mediante el intercambio de mensajes estructurados.

II. Componentes del sistema

1. Cliente

El cliente es la entidad que inicia la comunicación. Sus responsabilidades incluyen:

- Enviar solicitudes de consulta.
- Procesar y presentar las respuestas recibidas.

El cliente no interactúa directamente con los servidores de piezas. Todas las solicitudes son enviadas al intermediario, el cual actúa como punto único de acceso al sistema.

2. Servidor intermedio (intermediario)

El intermediario es el componente central del protocolo. Su responsabilidad principal es coordinar el flujo de comunicación entre el cliente y los servidores.

Cuando el intermediario recibe una solicitud, realiza las siguientes acciones:

- Interpreta el tipo de solicitud recibida.
- Determina si la solicitud requiere acceso a datos del sistema.
- Consulta su tabla de rutas para identificar el servidor adecuado.
- Reenvía la solicitud al servidor correspondiente.
- Procesa la respuesta recibida.
- Genera una respuesta final adecuada para el cliente.

3. Servidor

El servidor es responsable de almacenar y gestionar la información de las figuras. Ante una solicitud, el servidor:

- Verifica la existencia de la figura solicitada.
- Recupera la información correspondiente desde su almacenamiento.
- Construye una respuesta con los datos solicitados o con una indicación de error.

El servidor no interactúa directamente con el cliente, sino exclusivamente con el intermediario.

III. Formato de mensajes

Los mensajes en el protocolo se representan como estructuras compuestas por los siguientes campos:

- ***tipo:*** operación por realizar o respuesta generada.
- **ip source:** ip del que envía el mensaje, necesario en el JOIN Y HANDSHAKE.
- ***figura:*** nombre de la figura involucrada en la operación.
- ***figuras:*** lista de nombres de figuras
- ***segmento:*** mitad de la figura solicitada (1 o 2)
- ***piezas:*** listado de pares (pieza, cantidad) de la composición de una figura.

No todos los campos son utilizados en todos los mensajes; su uso depende del tipo de operación.

IV. Tipo de mensajes

El protocolo solicita un mínimo de acciones entre el intermediario con su servidor de piezas y los clientes, de esta manera se puede combinar con otros protocolos (btp,tinder Protocols ,Chamuy protocol) que manejan la interacción de cliente con intermediario, e intermediario con cliente.

1. ***Cliente - Intermediario***

   El cliente debe tener la capacidad solicitar figuras y la lista de figuras.

Tipo	*Descripción*
LIST_FIGURES	Solicita la lista de figuras disponibles
GET_FIGURE	Solicita una figura específica

2. ***Intermediario - Servidor***

   El intermediario debe tener la capacidad de solicitar figuras (como es una lógica multiprotocolo el intermediario puede pedir una en una o todas las disponibles) y la lista de figuras.

Tipo	*Descripción*
ASK_FIGURE	Consulta por una figura específica o todas

3. ***Servidor - Intermediario***

El servidor debe tener la capacidad de informar que la figura (o las figuras) solicitadas no está disponible, o en caso contrario dar toda la información de esta figura.

Tipo	***Descripción***
FIGURE_FOUND	Información de la figura está disponible
FIGURE_NOT_FOUND	Figura no encontrada / no existe

4. ***Intermediario - Cliente***

El intermediario debe tener la capacidad de responder con lo solicitado o con un mensaje de error si no es posible.

Tipo	***Descripción***
FIGURES_LIST	Retorna la lista de figuras disponibles
RETURN_FIGURE	Retorna la información de una figura
FIGURE_NOT_FOUND	Figura no encontrada

V. Respuestas

El protocolo define tres tipos principales de respuestas visibles para otro intermediario:

Respuesta de figura

El mensaje *RETURN_FIGURE* contiene:

- Nombre de la figura
- Segmento consultado
- Lista de piezas con sus cantidades

1. ***Respuesta de error***

El mensaje *FIGURE_NOT_FOUND* indica que la figura solicitada no está disponible o que la solicitud no pudo ser procesada correctamente, también puede especificar el tipo de errror.

VI. Modelo de datos

Las figuras se componen de dos segmentos independientes, cada uno formado por un conjunto de piezas.

Cada pieza está definida por su nombre, cantidad y metadatos asociados. De igual forma, cada figura puede incluir información adicional como fecha de creación o modificación.

VII. Bitácoras (Logs)

Las entidades del sistema generan registros asociados a los eventos de comunicación. Estos registros incluyen:

- Recepción de solicitudes
- Gestión y enrutamiento
- Generación de respuestas
- Ocurrencia de errores

El propósito de las bitácoras es proporcionar trazabilidad del sistema, facilitar el monitoreo de su comportamiento y apoyar procesos de depuración.

VIII. Tabla de Figuras del intermediario

El intermediario mantiene una estructura denominada **tabla de figuras** (o tabla de rutas), la cual permite asociar cada figura con uno o más servidores capaces de proveer su información.

El enrutamiento no se realiza únicamente mediante direcciones IP, sino a través de sockets, pertenecientes a la clase VSocket. Cada conexión se gestiona mediante un hilo independiente, el cual se encarga de escuchar continuamente el canal de comunicación, (similar a un *listen()*), procesar solicitudes y enviar respuestas.

Además, el intermediario mantiene una tabla de **figuras globales**, que contiene todas las figuras conocidas a través de sus conexiones con otros intermediarios. Esta tabla permite validar rápidamente si una figura existe en la red antes de iniciar una búsqueda más costosa.

Durante la ejecución del sistema no se agregan nuevas figuras dinámicamente; por lo tanto, cualquier consulta siempre resultará en una figura válida o en un ***FIGURE_NOT_FOUND.***

"House"

"Orchid"

"Fish"

"Giraffe"

Uso

Cuando el intermediario recibe una solicitud de una figura específica:

1. Consulta la tabla de figuras globales.
2. Si la figura no se encuentra en la tabla de figuras, retorna ***FIGURE_NOT_FOUND.***
3. De encontrarla, la busca en su propio servidor de figuras.
4. En caso de no estar en el servidor del intermediario, implementa un reenvío del request (fork() - multicast) hacia el resto de intermediarios con el tipo de paquete correspondiente.
5. Si se obtiene la figura del otro intermediario se vuelve a comunicar en el canal.
6. Si no la encuentra procede a eliminar de su tabla de figuras globales y retorna:

FIGURE_NOT_FOUND

En caso de existir múltiples servidores asociados a una misma figura, el intermediario selecciona el primero en responder a la solicitud del fork.

IX. Comunicación entre intermediarios

Conexión a la red

Cada intermediario se conecta a la red de intermediarios a través de un Switch centralizado (todos conocen el puerto y la ip de este switch). Tanto la conexión para el switch como para los intermediarios se maneja con el protocolo IPv4, sin encriptado SSL. Para crear la conexión:

- El intermediario envía una solicitud JOIN utilizando UDP.
- El switch realiza un reenvío (broadcast) hacia los demás intermediarios previamente registrados.
- Posteriormente, el switch almacena la dirección del nuevo intermediario.

Los demás intermediarios que reciben este JOIN, harán un HANDSHAKE con el intermediario que envió la solicitud, para establecer una conexión directa TCP/IP. Estas conexiones utilizan un rango de puertos distinto al de los clientes (por ejemplo, 2000–3000), de esta forma el intermediario recibe diferentes tipos de paquete dependiendo del puerto, para esta versión del protocolo el paquete de JOIN que recibe del switch y del HANSHAKE que recibe del resto de intermediarios es procesado por el mismo hilo.

Intercambio de información

Cuando un intermediario recibe un HANDSHAKE de otro intermediario, estos intercambian sus tablas de figuras, permitiendo actualizar sus tablas de figuras. Los intermediarios también pueden enviar y recibir paquetes, de tipo:

- ***INTERMEDIARY_REQUEST*** (solicitudes de figuras)
- ***INTERMEDIARY_RESPONSE*** (respuestas con datos de figuras)

Con cada HANDSHAKE que haga con otro intermediario, lo guardará en un vector de sockets, para que pueda ir preguntando secuencialmente intermediario por intermediario si no tiene la pieza que busca el cliente localmente. También estará escuchando cada canal que tenga con un intermediario si llega a tener una solicitud.

Manejo de conexiones

Por cada conexión entre intermediarios, se crea un hilo que maneje el canal (mediante el socket). De esta forma, se establece un canal distinto para cada comunicación.

El uso de hilos es necesario, ya que distintos intermediarios no pueden comunicarse a través de un mismo canal individual. Cada conexión requiere su propio canal independiente para evitar interferencias en la comunicación.

El hilo se encarga tanto de la lectura como de la escritura en su canal correspondiente con el intermediario específico. Además, procesa los mensajes que recibe: si se trata de un INTERMEDIARY_RESPONSE, envía el paquete al cliente; y si es un INTERMEDIARY_REQUEST, procesa la solicitud según corresponda.

Cuando un intermediario recibe la respuesta de una solicitud hecha por un cliente específico, este puede reconocer a dicho cliente mediante la dirección que contiene el paquete enviado tanto en el proceso de solicitud como en el de respuesta.

Definiciones de los paquetes

Paquete *INTERMEDIARY_JOIN*

- tipo uint8_t = **INTERMEDIARY_JOIN** : En el primer atributo se especifica que es un tipo de solicitud JOIN y busca conexión.
- source_ip 16 byte addr : Dirección Ip del intermediario.

Paquete *INTERMEDIARY HANDSHAKE*

- tipo uint8_t = **INTERMEDIARY_HANDSHAKE**
- content_length uint8_t
- content[] =(nombre de cada figura en lista partida por comas)

Paquete *INTERMEDIARY_REQUEST / INTERMEDIARY_RESPONSE*

- tipo uint8_t =
    - INTERMEDIARY_REQUEST: Especifica que es una solicitud de figura.
    - INTERMEDIARY_RESPONSE: Especifica que es un paquete de respuesta.
    - ERROR: la solicitud es incorrecta o hubo un error mientras se procesaba la solicitud.

- mitad int = 1 primera, 2 segunda, 3 toda.
- direccion_Socket uint8_t
- content_length uint8_t
- content[]=
    - nombre de la figura
    - nombre de la pieza de la figura, separado por comas

Nota: El socket en el content se refiere al objeto socket de clase VSocket, este contiene tiene la dirección ip, puerto de entrada, socketId (file Descriptor).

X. Manejo de errores

El protocolo contempla distintos escenarios de error y define comportamientos específicos para cada uno.

1. ***Figura no encontrada***

Si la figura solicitada no existe en la tabla de rutas o no puede ser obtenida desde el servidor, el sistema responde con *FIGURE_NOT_FOUND*.

2. ***Parámetros inválidos***

Si el cliente proporciona un segmento inválido, el sistema utiliza un valor por defecto (segmento 1) para garantizar la continuidad de la operación.

3. ***Inconsistencias en la tabla de rutas***

Si una figura está registrada en la tabla de rutas pero los servidores asociados no pueden responder, el intermediario puede intentar con otros servidores disponibles.

Si ninguna alternativa es válida, se retorna *FIGURE_NOT_FOUND*.

XI. Valores en tipo

ID_TIPO	Significado	Descripción	Cuándo pasa
300	INTERMEDIARY_REQUEST	es una solicitud	cuando se envia una solicitud
200	INTERMEDIARY_RESPONSE	el mensaje cumple con la estructura y los requerimientos	en una solicitud correcta sin malfuncionamiento del servidor.
101	INVALID_FORMAT	El mensaje no cumple con la estructura del protocolo.	Campos faltantes, separadores incorrectos.
102	DESTINATION_UNAVAILABLE	El nodo destino no existe o no es alcanzable.	Nodo desconocido o desconectado.
104	FIGURE_DOES_NOT_EXIST	La figura solicitada no existe.	Figura no registrada.
105	INVALID_HALF	La mitad solicitada no es válida.	Valores fuera de rango.
106	TIMEOUT	No se recibió respuesta en el tiempo esperado.	Falla de comunicación o nodo caído.
107	UNSUPPORTED_TYPE	El ID_TIPO no es reconocido.	Código inexistente.
108	INVALID_PARAMETERS	El cuerpo del mensaje es incorrecto.	Falta figura, mitad, etc.
110	OPERATION_NOT_ALLOWED	Operación inválida en el estado actual.	Ej. consultar sin registro previo.

XI. Limitaciones del protocolo

No existe una restricción además de los requisitos básicos entre cliente intermediario el intermediario servidor, pero la comunicación intermediario intermediario es muy estricta,

solo pueden pasarse las listas en un handshake y solo pueden solicitar una figura en un INTERMEDIARY_REQUEST.